- ***2*** Given an existing 3-frame state1 and a new single frame 2, adds the latest frame to the queue
- ***3*** Creates a state with three copies of the same frame and adds a batch dimension

8.5. Setting up the Q-network and policy function

As we mentioned, we will use a deep Q-network (DQN) for the agent. Recall that a DQN takes a state and produces action values, i.e., predictions for the expected rewards for taking each possible action. We use these action values to determine a policy for action selection. For this particular game there are 12 discrete actions, so the output layer of our DQN will produce a vector of length 12 where the first element is the predicted value of taking action 0, and so on.

Remember that action values are (in general) unbounded in either direction; they can be positive or negative if the rewards can be positive or negative (which they can be in this game), so we do not apply any activation function on the last layer. The input to the DQN is a tensor of shape `Batch x 3 x 42 x 42`, where, remember, the channel dimension (`3`) is for the most recent three frames of game play.

For the DQN, we use an architecture consisting of four convolutional layers and two linear layers. The *exponential linear unit* (ELU) activation function is used after each convolutional layer and the first linear layer (but there's no activation function after the last linear layer). The architecture is diagrammed in figure 8.12. As an exercise you can add a *long short-term memory* (LSTM) or *gated recurrent unit* (GRU) layer that can allow the agent to learn from long-term temporal patterns.

Figure 8.12. The DQN architecture we will use. The state tensor is the input, and it is passed through four convolutional layers and then two linear layers. The ELU activation function is applied after the first five layers but not the output layer because the output needs to be able to produce arbitrarily scaled Q values.